

TDT-v5.5

離散状態遷移学習理論

候補評価エンジンの高速化とE17a再現検証
CUDA Graph版の処理方法・数値一致性・性能・監査

実験終了版・2026年9月8日

結果の要点	今回確認できたこと
CPUの完全一致最適化	復元キャッシュ1.107倍、候補コピー削減1.013倍。検査範囲でFP32ビット一致。
GPUの速度	16残差ブロック・100区間×3 seedでCUDA Graph版15.127倍（CPU復元キャッシュ比）。
E17aの全長再現	8残差ブロック・12,000区間×3 seed：90.587 ± 0.398%。既存90.637 ± 0.430%との差−0.050 pt、精度許容基準に合格。
一致性と終了状態	GPUはCPUと発火が分岐。元のCPU fastは遅くなり、seed 1・2を途中終了。完全一致と精度維持を区別する。

v5.4以降の実験を追加し、後半にv5.4全文とその添付資料を継承する。追加部は確定結果・途中終了記録・実行時ソースに基づく。PDFのしおりから各節および旧版へ移動できる。表の±は3 seedの標本標準偏差。ptは精度のパーセントポイント。

読者案内・共通条件

読む目的	参照箇所
全体結果	終了報告の結論、1〜4節、精度と速度の図
CUDA Graphの実装詳細	処理図、G1〜G8：データ配置、乱数、BMM、捕捉・再生、同期、判定
検証と追跡	監査・添付資料一覧、埋め込みZIP内のCSV・設定・実行時ソース
v5.4以前の科学実験	付録のTDT-v5.4全文：E17〜E20と過去版
固定した項目	内容
データ	MNIST、9×10平均プーリングで90次元。(x/255−0.1307)/0.3081。train 10,000 / val 1,000 / test 10,000、data_seed=0。
TDT	閾値8、選択16辺、K=64候補対、batch128、最大1発火/区間、C8、leak1、区間末リセット。S初期0.02、EMA0.1、下限1e-5。全長再現は12,000区間。
重み・構造	INT8三値、初期ゼロ率1/3、gain1、固定スケール1/sqrt(fan-in×2/3)、バイアスなし。pre-norm残差、RMSNorm eps1e-8、ゲインなし。
精度・乱数	演算・復元・ストリーム・logits・損失FP32、A8 absmax/127。seed0/1/2、batch_seed=seed+100000、CPU threads1。GPUで逆伝播・STEは使わない。

短期ベンチマークは保存済みE18a重みから各100区間の動作測定であり、新規の収束実験として数えない。全長GPU再現はE17a初期状態から3 run。元のCPU fastは1 run完了・2 run途中終了。旧E17/E18/E20対照は再学習していない。

2026-09-08。ユーザーの指示により実験を終了した。対象はCPU fastエンジン、CPUメモリ処理の単独最適化、GPU候補並列化・CUDA Graph、およびGPUによるE17a再現検証。残っていたCPU fast学習と待機中の集計プロセスは停止済み。未完了seedへのtest評価や追加学習は実施しない。

結論

- CPUで計算結果を保持した高速化は、復元済みFP32重みの再利用で 1.107倍、候補重みの全体コピー削減だけでは 1.013倍。検査した全候補・学習区間でビット一致した。

- CPUのプレフィックスキャッシュ + 低ランク補正 + 候補バッチ化は、A8丸め付近の数値差を解消する再評価によって 約2倍遅くなった。3 seedの完全再現検証は途中終了。

- GPUでは候補並列化が有効で、CUDA Graph版は16残差ブロックの短期測定でCPU復元キャッシュ比 15.127倍。逐次forwardをGPUに移すだけでは遅くなった。

- CUDA Graph版のE17a全長再現は3 seedとも完了。test $90.587 \pm 0.398\%$ 、既存E17a $90.637 \pm 0.430\%$ との差は -0.050 ポイント。事前登録した平均精度 ± 0.3 ポイントの基準を満たした。ただしCPUとの数値・発火系列の完全一致は満たさない。

以下の $\pm$ は3 seedの標本標準偏差。速度倍率は同じ表内の基準時間との比であり、異なる深さや測定条件の倍率を混同しない。

1. 元のCPU fastエンジン

E17a相当：8残差ブロック、幅76、18行列、100,016三値重み、A8 + ReLU。既存の候補生成・乱数消費・投票・カウンタ・発火判定を共通コードで使用した。

実装前提の調査では、既存の64候補対は対ごとにミニバッチが異なり、16辺の摂動は複数行列にまたがっていた。そのため、単一共通バッチ・単一行列の摂動という単純化は採用できず、候補ごとのバッチと全被摂動行列への補正を維持した。

固定重み検査の未補完fast版は最大相対損失誤差 $7.64477e-4$ で、要求の $1e-5$ 未満を満たさなかった。A8丸め境界に近い候補を元のnaive計算で再評価する補完版は $1.77193e-7$ 、投票・カウンタ・発火の不一致0となった。ただし全長実験の記録済み全区間で128候補すべてが再評価対象となり、高速化を打ち消した。これは検査範囲での一致であり、任意入力に対する数学的証明ではない。

構造	naive 秒/区間	補完fast 秒/区間	速度倍率
8残差ブロック・100区間	0.166252	0.345763	0.481倍
16残差ブロック・100区間	0.295492	0.651013	0.454倍

深さ2倍に対する時間増加はnaive約1.78倍、fast約1.88倍。8ブロックのnaive 12,000区間ベンチマークは1,970.70秒（32.85分）。補完fast seed 0の学習は4,289.91秒（71.50分）だった。ただし後者にはvalidation等を含み、後続実験との資源共有もあり、両総時間の比を厳密な専有条件の速度倍率とはしない。

100区間ベンチマークのプロセス最大RSSは8ブロックでnaive約788.9 MiB、fast872.4 MiB、16ブロックで789.3 MiBと995.4 MiB。データ読みやアロケータ保持も含むため、差をそのまま一時キャッシュの厳密なピークとは解釈しない。8ブロックの論理重みは100,016 bytes、論理カウンタは1,000,160 bytes。テンソル参照からのキャッシュ推計は約102.9 MiBだが、参照の重複を含む。単純なforward換算は実装の異なるミニバッチと全候補再評価を含めないため、速度予測にならなかった。

終了時点

seed	記録済み区間 / 12,000	保存重みの区間	最新validation	最終test
0	12,000	12,000	90.1%	91.11%
1	5,282	5,000	85.9%	未評価
2	5,465	5,000	85.9%	未評価

seed 0は既存E17aと最終重み・test精度が一致し、発火分岐なし。停止時監査では3 seedそれぞれの記録済み区間まで、候補絶対損失差配列および共通学習指標の既存記録との差は0だった。部分seedの保存重みは5,000区間時点であり、最後のログ区間の重み保存を主張しない。事前確保されたabs_y.npyは記録済み区間までのみ有効。CPU fastの3 seed平均と最終合否は未判定。

記録：停止監査（results/fast-engine-e17a-20260908/stop_audit.json）、区間一覧（results/fast-engine-e17a-20260908/partial_results.csv）、停止記録（results/fast-engine-e17a-20260908/stop_record.json）。

2. CPU：復元再利用とコピー削減の独立測定

16残差ブロック・幅76・34行列・192,432重み。保存済みE18aの各seedから100区間、各条件3 seed、CPU threads=1。二つの変更を別々に測定し、組合せ条件は実施していない。

条件	秒/区間	naive比速度	時間削減
naive	0.301504 ± 0.001052	1.000倍	—
復元済み重み再利用	0.272359 ± 0.001099	1.107倍	9.67%
候補全体コピー削減	0.297522 ± 0.002516	1.013倍	1.32%

4,608候補損失比較でFP32ビット一致。投票・カウンタ・発火・S・乱数状態も一致し、100区間後の重みも全条件で一致した。コピー削減の効果は小さく、測定揺らぎへの注意が必要。test評価と12,000区間学習は行っていない。

詳細：単独最適化レポート（[results/allocation-ablations-16blocks-20260908/README.md](https://github.com/tdt-project/tdt/blob/main/results/allocation-ablations-16blocks-20260908/README.md)）。

3. GPU：同規模の100区間測定

RTX 5090、FP32、TF32無効、CPU threads=1。16残差ブロック、3 seed×100区間。CPUの候補生成・判定、転送、GPU計算、受理更新を含む区間時間。初期転送・ウォームアップ・Graph構築は別記録。

条件	秒/区間	CPU復元キャッシュ比
CPU復元キャッシュ	0.271573 ± 0.001365	1.000倍
GPU逐次	0.613028 ± 0.002405	0.443倍
GPU候補並列	0.021019 ± 0.000054	12.920倍
GPU候補並列 + CUDA Graph	0.017952 ± 0.000080	15.127倍

GPU逐次では小さい演算の呼出し負荷が大きく、候補をまとめて演算することで高速化した。GPU処理区間は並列版5.188 msからGraph版2.142 msへ短縮したが、CPU側の候補生成・判定は残るため、区間全体は約17.95 msとなる。

GPU予約メモリ最大値は逐次150 MiB、並列170 MiB、Graph224 MiB。Graphの追加予約は並列比54 MiB。これらはPyTorchの予約量で、CUDAコンテキストを含むプロセス全体のVRAMではない。Graphの割当カウンタは専用プール再生時の一時メモリを直接表さないため、予約量で比較する。

CPU対GPUの最大相対損失誤差はGraph版 0.00279344 で、1e-5基準に不合格。投票・発火系列にも差が生じた。一方、GPU並列版とGraph版は検査した固定入力6ケースと100区間の全seedで同一だった。A32対照ではCPUとの差が約3e-7以下となり、FP32演算順序の微差がA8丸めを通じて増幅することと整合する。CPU互換の完全一致高速化とは結論しない。

詳細：GPU短期ベンチマーク（results/gpu-evaluation-16blocks-20260908/README.md）。

4. CUDA Graph版：E17aの12,000区間×3 seed

比較対象をE17aに合わせ、8残差ブロック・幅76・100,016重みを初期状態から学習。TDT設定・データ分割・乱数規則を維持した。validationは初期と500区間ごと、testは最終時のみ既存と同じCPU推論で評価した。testを条件調整に使っていない。

seed	既存E17a test	GPU学習 test	差（ポイント）	最初の発火分岐区間	GPU学習秒
0	91.11%	91.03%	-0.08	4	194.36
1	90.27%	90.47%	+0.20	21	197.17
2	90.53%	90.26%	-0.27	10	196.79
平均±標準SD	90.637 ± 0.430%	90.587 ± 0.398%	-0.050	—	平均196.10

- 発火系列完全一致：不合格。

- 事前登録したtest平均90.637±0.3%（90.337～90.937%）：合格。

- CPUに対する固定損失の厳密な数値一致：不合格。精度許容幅の合格と区別する。

平均学習時間は3分16秒/run。既存CPU E17aはseed順2,191.03 / 2,159.95 / 2,197.55秒、平均36分23秒/runで、記録上の比は約11.13倍。実行時の資源共有条件が異なるため、同時条件の性能測定には上の16ブロック表を用いる。

独立監査は77ハッシュ検査、全区間の候補損失・S・乱数消費・発火からの最終重み再構成、選定区間のGPU損失再現などを通過。全seedの最終testは既存記録を使用し、監査でtestの再評価は行わなかった。

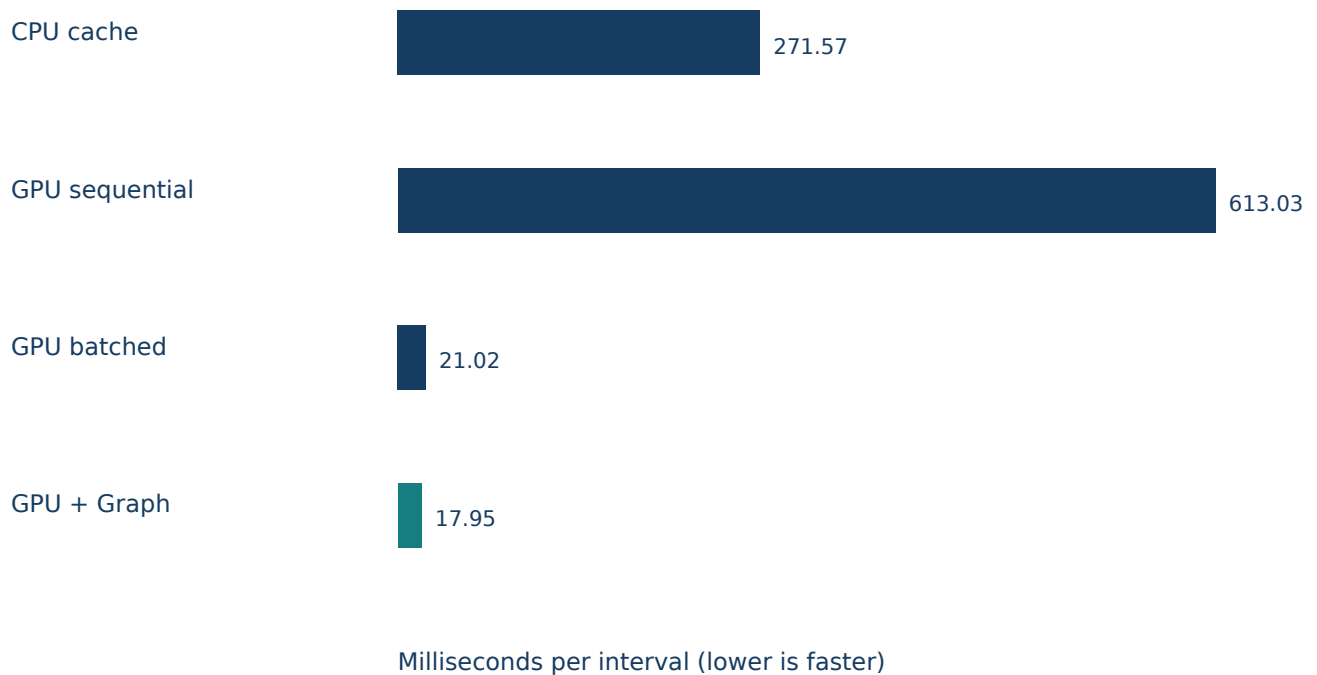
詳細：確定集計（[results/gpu-e17a-reproduction-20260908/report.json](#)）、全記録（[results/gpu-e17a-reproduction-20260908/](#)）。

成果物と解釈の範囲

実装はfast_engine.py、allocation_engines.py、gpu_evaluation_engines.pyおよび各runner/auditスクリプト。既存naive学習則は維持し、別エンジンとして追加した。各結果ディレクトリに設定、CSV、実行時ソース、ハッシュ、資源共有記録を保存。途中終了CPU実験には停止時の追加SHA-256マニフェストを保存した。

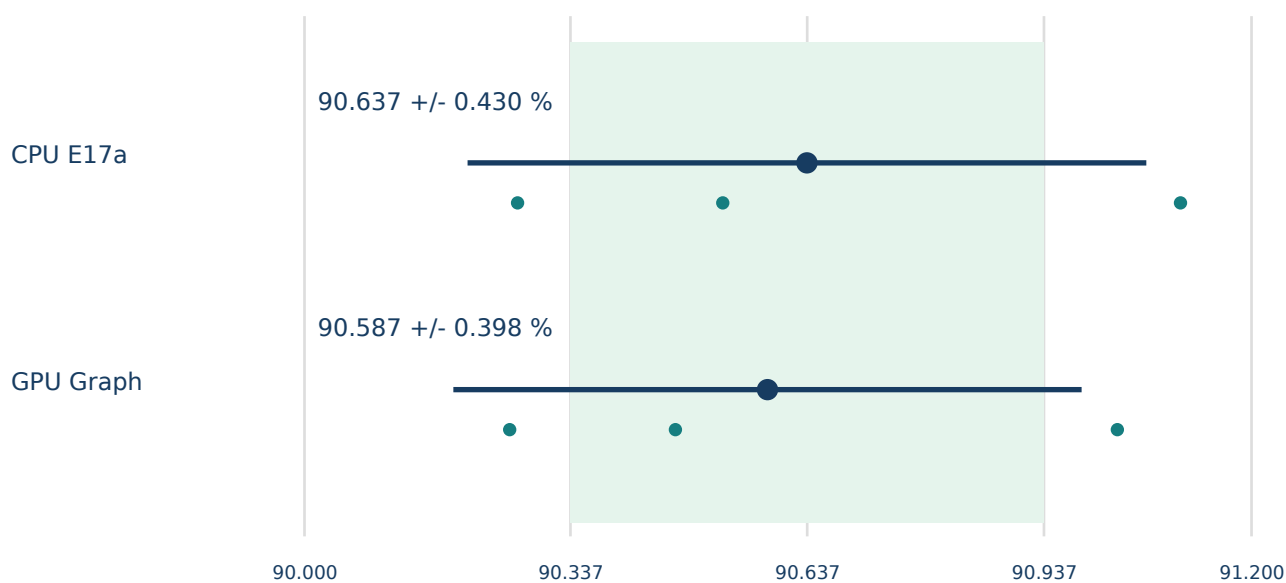
得られた実務上の選択肢は、検査範囲でCPU結果をそのまま保つ復元キャッシュと、CPUからの学習軌跡の分岐を許容して最終精度を維持したGPU CUDA Graph版である。後者の精度確認はE17aの3 seedに限られ、他の深さ・データ・seedへの一般化は今回の結果から保証しない。追加実験は実施せず終了する。

図1. 16残差ブロックの区間時間



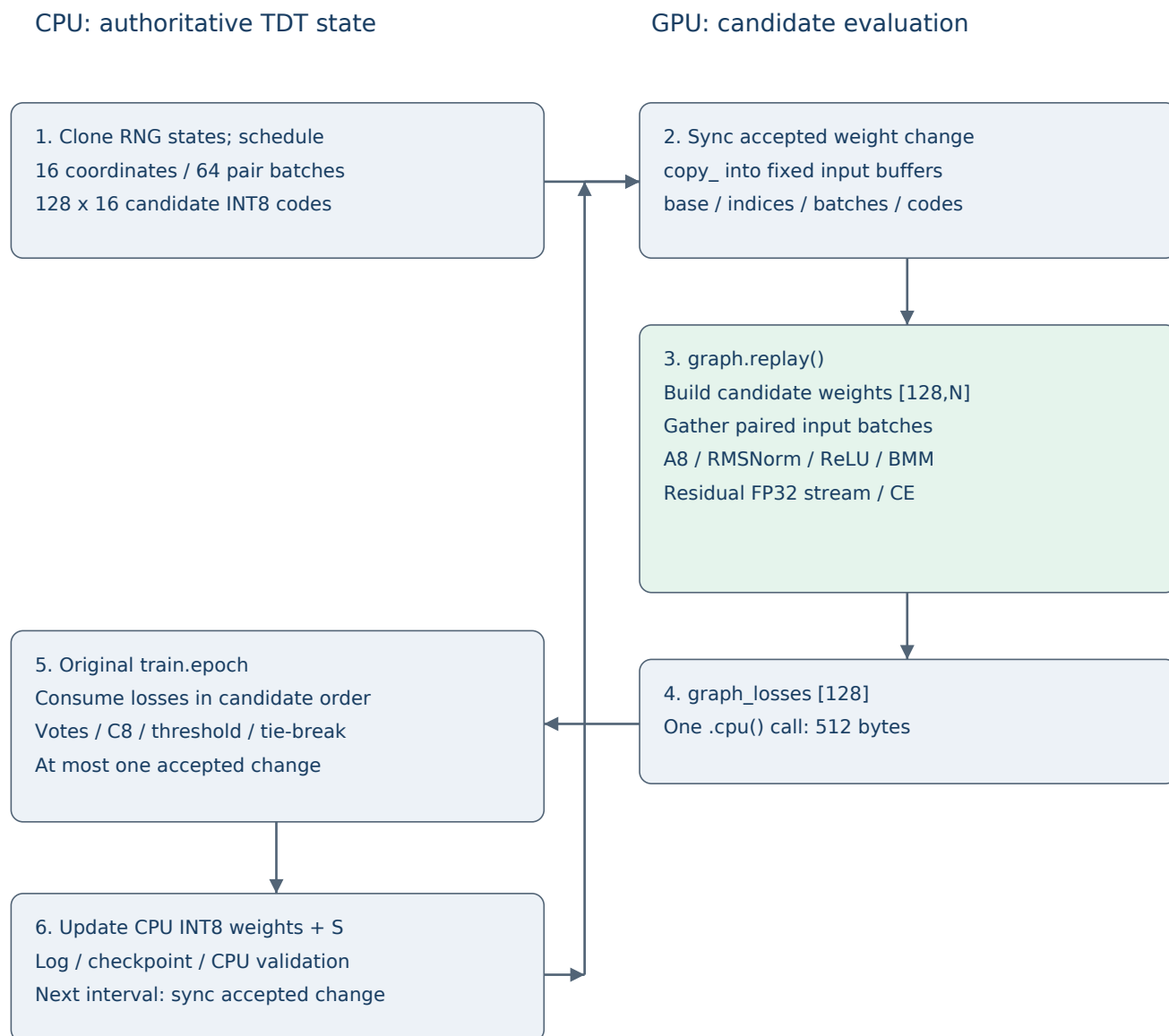
3 seed平均。CPU候補生成・判定、転送、GPU候補評価、受理更新を含む。Graph捕捉・ウォームアップは別。CPU復元キャッシュ比15.127倍。GPUに逐次移すだけでは遅くなった。元のCPU fastベンチマークとは異なる比較基準。

図2. E17a全長再現の最終精度



大点と横線は平均±標本SD、小点はseed値。背景帯はGPU平均の事前登録許容範囲90.337～90.937%。個別seedに課す基準ではない。平均差-0.050 ptで合格だが、統計的同等性やCPUとの発火系列完全一致を示すものではない。

図3. CUDA Graph版の1区間の処理



Captured: GPU green box only. RNG and decisions remain on CPU.

Graph捕捉は開始時に実施。区間ごとには固定バッファへ内容をコピーし、同じGPU処理を再生する。図のCPU損失消費では正本Generatorが元のコードどおり進む。先読みGeneratorとは分離されている。

G1. 最適化した部分と維持した部分

本節の一次資料はgpu_evaluation_engines.pyとrun_gpu_e17a.pyの実行時ソースである。説明対象は今回実行したgpu_graphであり、提案のみの方式とは区別する。GPU短期測定の16残差ブロック（34行列）と全長E17a再現の8残差ブロック（18行列）は、同じ評価器を構造情報で切り替えて用いた。幅はともに76。

CPU上のINT8三値重みを学習状態の正本とする。GPUには学習データ、三値コード、固定スケール、復元済みFP32重みを常駐させる。GPUが返す128個の候補損失を、既存train.epochと同じ候補順にCPUへ渡す。投票・C8・leak・タイブレーク・発火・S更新は元のCPUコードを使う。逆伝播、STE、Adam、潜在FP32学習重みは導入しない。

CPU fastの低ランク出力補正とは別の方式である。GPU版は候補ごとの完全な重み行列を構成し、全層のforwardをBMMで評価する。プレフィックス省略、低ランク出力補正、CPUでの候補損失再評価を使わない。高速化は候補並列化とGPU演算の投入負荷低減によるもので、論理的な候補数を減らしてはいない。

G2. データ配置とテンソル形状

候補数 $P=128$ 、候補対数 $K=64$ 、評価例数 $B=128$ 、幅 $d=76$ 。Nは三値重み総数。行列は実装のF.linearに合わせ[out_features, in_features]で保持する。

所在	名前・形状	型・用途
CPU	model.weights [N]	INT8。受理された三値状態の正本
CPU	g / bg、S、カウンタ	候補・確率丸め用乱数、バッチ用乱数、証拠の集約
GPU	x [10000,90] / y [10000]	FP32学習入力とINT64ラベル。初期化時に転送
GPU	codes [N] / scales [N] / base [N]	INT8コード、FP32固定層スケールを座標に展開、FP32復元重み
GPU固定入力	indices [16]	INT64。区間で選択された全体座標
GPU固定入力	batches [64,128]	INT64。各候補対のミニバッチ添字
GPU固定入力	candidate_codes [128,16]	INT8。候補ごとの選択座標の三値コード
GPU一時値	weights [128,N]	FP32。128候補分の完全な有効重み
GPU一時値	x [128,128,90] / h [128,128,76]	候補×例×特徴。残差ストリームはFP32
GPU固定出力	graph_losses [128]	FP32。候補ごとの128例平均交差エントロピー

N=100,016のE17aでは、候補重みテンソルだけで $128 \times N \times 4 = 51,208,192$ bytes (48.84 MiB)。16ブロックでは98,525,184 bytes (93.96 MiB)。入力バッチは5.625 MiB、幅76のストリーム1本は4.75 MiB。この計算値は単一テンソルの容量であり、実測VRAMピークではない。量子化中間値、複数の活性、ライブラリ作業領域、Graph専用プールも必要となる。

三値重みをFP32へ復元するのは、固定スケールを掛けた有効重みで既存と同じFP32線形演算を行うためである。GPU版もINT8行列積や三値専用カーネルではない。FP32のbaseはコードから再生成可能な計算用キャッシュで、勾配更新を蓄積する潜在重みではない。

G3. CPUで候補スケジュールを先読みする

1区間の冒頭で、候補用gとバッチ用bgの状態を複製したローカルGeneratorを作る。正本Generatorはこの段階で進めない。gの初期seedはseed+1、bgはseed+100000。初期重みは元のseed 0/1/2で生成する。

複製gのrandperm(N)から16座標を選ぶ。64候補対それぞれで、複製bgから128例の添字を生成し、元のcandidate_pair由来のコードでplus/minus候補を作る。GPU転送用には選択した16座標の三値コードだけを保存する。候補順はplus_0, minus_0, plus_1, minus_1, ...で固定する。

各候補対の後でtorch.rand([16], generator=g)を消費し、元のaccumulateの確率的丸めで使われる乱数消費も先読み時に再現する。これを省くと、次の候補生成の乱数がずれる。先読みは乱数消費数が固定された今回の学習コードに対応しており、将来の判定コード変更に自動追従する保証はない。

その後、正本のg/bgを使って元のtrain.epochを実行する。候補生成と乱数消費は従来どおり行い、loss関数だけが先に計算したGPU損失を順に返す。候補を先読みする計算はCPUで重複するが、乱数・判定経路を維持するための構成である。trace検査では渡された候補コードとミニバッチを先読み結果に照合し、最後に128損失すべてが消費されたことを確認する。

G4. GPU上で128候補のforwardを構成する

prepared()はbase.expand(128,-1).clone()で128候補分の有効重みを作り、weights[:, indices]をcandidate_codes.float()×scales[indices]で上書きする。FP32差分の加算ではなく、三値コードと固定スケールの積を直接代入する。選択された16座標が複数行列にまたがっても、この全体座標方式で反映される。

データはx[batches]で[64,128,90]に集め、repeat_interleave(2,dim=0)で[128,128,90]とする。同じ候補対のplus/minusは同じ128例を使い、異なる候補対のバッチは元の系列どおり異なる。ラベルも同じ対応で複製する。

各層では候補重みの該当区間を[128,out,in]にviewし、torch.bmm(Q(x), W.transpose(1,2))を実行する。すべての候補が同じ重みを共有する単一の大きなmatmulではなく、候補ごとに異なる行列を使うBMMである。候補軸Pを保つことで128通りの全forwardを並列評価する。

入力射影 : $h = \text{BMM}(Q_A8(x), W_in^T)$ 。各ブロック : $u = \text{RMSNorm}(h)$ 、 $a = \text{BMM}(Q_A8(u), W1^T)$ 、 $b = \text{BMM}(Q_A8(\text{ReLU}(a)), W2^T)$ 、 $h = h + b$ 。出力 : $\text{logits} = \text{BMM}(Q_A8(\text{RMSNorm}(h)), W_out^T)$ 。正規化と量子化は最後の特徴次元だけで集約し、候補間・例間を混ぜない。

RMSNormは $x / \sqrt{\text{mean}(x^2, \text{dim}=-1)+1e-8}$ 。A8は各候補・各例・各点でabsmax/127を計算し、round(x/scale)を[-127,127]へクリップしてINT8化後FP32に復元する。丸めはties-to-even。全ゼロ行ではscale=1、正の極小scaleには元コードのFP32 tiny下限を適用する。ストリーム自身は量子化しない。

logits [128,128,10]を[16384,10]にreshapeし、reduction='none'の交差エントロピーを計算する。[128,128]へ戻し例軸だけをmean(1)することで、順序を保持した128候補損失を得る。候補間で平均しない。

G5. CUDA Graphの捕捉、固定バッファ、再生

GPUEvaluatorの初期化時に、データ・重み・固定入力バッファをGPUへ確保する。gpu_graphモードでは別CUDA streamでparallel()を3回ウォームアップし、stream間の待機とcuda.synchronize()で準備を完了する。

続いてtorch.cuda.CUDAGraph()を作り、with torch.cuda.graph(self.graph): の中で一度parallel()を呼ぶ。この呼出しはprepared()による候補行列・バッチ構成、A8、RMSNorm、ReLU、残差加算、BMM、交差エントロピーまでを捕捉し、graph_lossesという出力テンソルを保持する。

各区間ではindices.copy_、batches.copy_、candidate_codes.copy_で同じ入力バッファの内容を更新してからgraph.replay()を呼ぶ。Graphが参照するバッファを別のテンソルへ差し替えない。候補やバッチが変わっても、形状と参照先を維持して内容を更新するため、再捕捉なしで再生できる。候補重みのcloneをコード上から除去するのではなく、捕捉時に用意したGraph用メモリを再利用する。

Graph内で再生する処理	Graph外で各区間行う処理
GPU上の候補重み展開・16座標代入	CPUの乱数先読み・候補生成
GPU常駐データから候補別バッチ構成	受理済み重みの同期、固定入力へのcopy_
全層のA8・RMSNorm・BMM・ReLU・残差加算	CUDAイベント計測と損失のCPU転送
全候補のFP32交差エントロピー例平均	元のCPU epochによる投票・C8・発火・S更新
graph_lossesへの出力	validation、checkpoint、ログ、最終test

Graphは学習ループ全体、CPU乱数、分岐する判定ロジックを捕捉していない。固定P=128、K=64、B=128、16座標に合わせた実装であり、これらの変更は本実験の対象外。CUDA GraphはGPU演算を一つの数学演算へ融合する仕組みとして使ったわけではなく、同じGPUワークフローの繰り返し投入をまとめている。

G6. 重み同期、転送、CPU判定への復帰

`sync_weights()`はCPU上に持つ前回同期時の重みコピーと正本を比較し、変わった座標だけをGPUへ送る。`codes`と`base`の該当座標を更新し、CPU側の同期用コピーも更新する。今回の発火上限は1なので、通常は0または1座標の変更となる。GPUの`base`更新も`code.float()×scale`の直接代入であり、丸め誤差を蓄積する差分加算ではない。

区間開始の同期は、前の区間末にCPUで受理された更新を次のGPU評価へ反映する。最終区間の更新後にGPUコピーを改めて同期する必要はなく、保存モデルと最終評価にはCPUの正本を使う。GPU上の乱数生成や独自の発火判断はない。

固定入力の内容量は`indices` 128 bytes、`batches` 65,536 bytes、`candidate_codes` 2,048 bytes、計67,712 bytes/区間（約66.1 KiB）に、受理更新の座標・コードが加わる。これはテンソルの論理ペイロードで、転送呼出しの内部費用を含む実測帯域ではない。学習入力全体や候補行列全体を区間ごとにCPUから送る方式ではない。

`graph.replay()`後、`graph_losses.cpu()`で128個のFP32損失（512 bytes）をまとめてCPUへ戻す。このCPU転送で結果を待ち、有限値を確認する。戻った損失を`plus/minus`順に既存`loss`呼出しへ渡すことで、元のCPUコードが差、確率丸め、カウンタ、発火、`S`を計算する。GPUイベントの開始は重み同期・入力更新の後、終了はGPU評価後に記録するため、イベント時間は区間全体の時間とは異なる。

この方式でもCPUのバッチ「添字生成」とTDT判定は必要である。CPUで128候補の`forward`を逐次計算する必要はなくなるが、CPU処理自体がなくなるわけではない。また、今回の`validation・test`は比較条件を揃えるためCPU推論を使用する。

G7. 再現性、速度、数値差の読み方

GPU設定はFP32 IEEE、TF32無効、torch.use_deterministic_algorithms(True)、CUBLAS_WORKSPACE_CONFIG=:4096:8。実行記録はPyTorch 2.13.0+cu130、PyTorch CUDA 13.0、driver 580.105.08、RTX 5090。nvidia-smiのCUDA Version 13.2表示と、PyTorchビルドのCUDA 13.0を区別する。

決定的なGPU実行を設定してもCPUとのFP32演算順序までは一致しない。行列積・損失の微差でA8の丸め境界をまたぐと量子化後の差が大きくなり、損失差・S・確率投票・発火へ伝わりうる。A32対照では最大相対損失誤差が約 $3e-7$ 以下だったのに対し、GPU並列/GraphのA8検査では0.00279344となった。この診断は丸めによる増幅と整合するが、全差異の単一原因を証明したものではない。

GPU並列版とGraph版は、初期/学習済み×3 seedの入力更新検査6ケースで128損失がビット一致し、100区間後の重み・S・乱数状態・発火も一致した。これに対しCPUとは発火が分岐する。CPU互換性、GPU eagerとGraphの同一性、最終testの許容幅は三つの別の検証事項である。

16ブロックのGPU処理区間は逐次596.209 ms、並列5.188 ms、Graph2.142 ms。CPU処理・転送等を含む全区間はそれぞれ613.028 / 21.019 / 17.952 ms。GraphによるGPU処理区間の短縮は約2.42倍だが、並列版から区間全体への上積みは約1.17倍である。CPU復元キャッシュ全区間271.573 msに対しては15.127倍。異なる時間範囲の倍率を混同しない。

Graphは128個の候補forwardに相当する全層の計算を残す。8ブロックでは18回の候補BMM、16ブロックでは34回の候補BMMがワークフローに含まれるが、各BMMは128候補を処理するため、「forward回数が128から1へ減った」と解釈しない。FLOPs削減と、並列実行・投入負荷削減による実時間短縮を区別する。

G8. 実装箇所と検証の対応

実装箇所	処理	検証記録
gpu_evaluation_engines.py / schedule	RNG先読み・候補順・バッチ対応	短期validationのindices/rng一致、全長audit
GPUEvaluator / prepared, parallel	候補別完全行列、A8、BMM、候補別損失	CPU/GPU相対誤差、A32対照、全候補loss CSV
GPUEvaluator / __init__, inputs, evaluate	捕捉・固定バッファ更新・replay	graph_input_refresh.csv、GPU eagerとのビット一致
GPUEvaluator / sync_weights	受理座標の同期・復元キャッシュ更新	更新後Graph入力検査、学習後重み監査
gpu_evaluation_engines.py / epoch	元のtrain.epochへ128損失を供給	候補・バッチtrace、消費数128、発火記録
run_gpu_e17a.py / worker	12,000区間・CPU評価・記録	seed別checkpoint、分岐記録、test各1回
audit_gpu_e17a.py	独立監査	全区間loss差/S/RNG、重み再構成、15区間のGPU再評価

上表のファイルと関数は、PDF添付資料の実行時sourcesに収録する。GPU短期測定と全長再現で保存したgpu_evaluation_engines.pyが同じ内容であることをPDF生成時にも確認する。報告書作成にあたって新たな学習やtest再評価は行わない。

計測範囲と全長runの時間内訳

seed	Graph準備 秒	engine 秒	validation 秒	naive分岐確認 秒	学習ループ 秒
0	0.440	187.351	1.379	0.675	194.360
1	0.457	187.322	1.378	3.511	197.166
2	0.465	188.747	1.385	1.673	196.788

engine時間はgpu_epoch呼出し全体。学習ループ時間は初期準備後から12,000区間終了までで、validation・途中のnaive比較・ログ・checkpointを含む。初期/最終の層単独プローブ、Graph準備、最終testはこの学習ループ時間の外。上表は相互排他的な全実行時間の分解ではないため、行の値を単純加算しない。

同一マシン・CPU threads1の短期比較でも、先行CPU実験との共有資源がある。GPU系列のCPU affinityは15。既存CPU E17aの過去時間との比11.13倍は参考値であり、同一負荷下の専有速度保証ではない。各rootのruntime_workers.jsonとoverlap記録を添付する。

監査・添付資料と版の継承

実験記録	PDF生成時に再照合した成果物ハッシュ数
allocation-ablations-16blocks-20260908	38
gpu-evaluation-16blocks-20260908	53
gpu-e17a-reproduction-20260908	84
fast-engine-e17a-20260908	77

GPU全長再現の既存独立監査は77ハッシュ、3 seed全36,000区間の損失差・S・乱数状態・発火からの最終重み再構成などを確認した。ここでのPDF生成時ハッシュ照合数は、各成果物マニフェストの登録ファイル数であり、独立監査の77件と対象・数え方が異なる。PDF生成では学習やtestの再評価をせず、保存済み3 seedから平均と標本SDを再計算した。

添付のTDT-v5.5_追加実験データ.zipは本文原稿、生成スクリプト、集計CSV、設定、監査、事前登録、停止記録、実行時sourcesを含む。各ファイルのSHA-256はsource_hashes.json。大容量の逐区間CSV、候補損失NPY、モデルPTは実験結果ディレクトリに保存済みで、ZIPには含めない。これらの追跡用マニフェストは添付する。

tdt_mnist/results/fast-engine-e17a-20260908

tdt_mnist/results/allocation-ablations-16blocks-20260908

tdt_mnist/results/gpu-evaluation-16blocks-20260908

tdt_mnist/results/gpu-e17a-reproduction-20260908

v5.4の165ページを後半へ継承し、本文・図の画素と埋め込み資料の一致を検査する。旧版の日付・結論・ページ番号は当時の記録として読む。旧版でFP32演算を用いていたことと、今回GPUで候補を並列評価したことを区別する。新規の科学条件や追加test探索は行っていない。